# Document technique

# Sommaire :

**Documentation technique X3COM** ..... 3
**1. Vue d'ensemble** ..... 3
**2. Stack technique** ..... 3
**3. Architecture logique** ..... 3
**4. Sécurité et contrôle d'accès** ..... 4
**5. Authentification et session** ..... 5
**6. Gestion compte utilisateur** ..... 7
**7. Mot de passe oublié et réinitialisation** ..... 7
**8. Commandes, paiement, facture PDF** ..... 7
**9. Prise de rendez-vous (RDV)** ..... 9
**10. Module admin** ..... 10
**11. Contenus Saviez-vous** ..... 11
**12. Modèle de données (principales tables)** ..... 13
**13. Observabilité et diagnostic** ..... 14
**14. Flux critiques (résumé)** ..... 14
**15. Points de vigilance techniques** ..... 15

# Documentation technique X3COM

Ce document décrit les points techniques importants de l'application, avec des extraits de code représentatifs.

## 1. Vue d'ensemble

X3COM est une application web e-commerce et prise de rendez-vous conçue autour d'une séparation claire entre interface, logique métier et persistance.
L'application s'appuie sur quatre briques techniques complémentaires:

- Un frontend Angular (standalone components) qui gere l'experience utilisateur.
- Un backend Node.js/Express qui applique les regles metier et expose les APIs.
- Une base Supabase PostgreSQL qui stocke les donnees fonctionnelles.
- Des services externes (Stripe pour le paiement, Resend pour l'email transactionnel).

La structure du repository reflete cette organisation:

- Frontend: projet-final/angular.
- Backend: projet-final/x3com-backend.
- Migrations base de donnees: projet-final/x3com-backend/migration_supabase.sql.

## 2. Stack technique

La stack est choisie pour repondre a trois besoins: rapidite de developpement, securite des echanges et maintenabilite du code.

**Frontend**

- Angular standalone pour limiter le couplage entre modules.
- HttpClient + interceptor JWT pour centraliser les appels API securises.
- Signals Angular (signal, effect) pour la gestion d'etat locale reactive.
- Router Guards (auth/admin) pour proteger les pages selon le role utilisateur.

**Backend**

- Node.js + Express pour exposer des routes REST lisibles et testables.
- @supabase/supabase-js pour l'acces type a PostgreSQL via Supabase.
- jsonwebtoken + bcryptjs pour l'authentification et le hash des mots de passe.
- stripe, resend, pdfkit pour le paiement, l'email transactionnel et la facture.
- helmet + express-rate-limit + cors + multer pour renforcer la securite HTTP.

## 3. Architecture logique

L'architecture logique separe les responsabilites entre le client Angular et l'API Express afin de garder un code previsible et evolutif.

### 3.1 Frontend (Angular)

- Le routage central (angular/src/app/app.routes.ts) definit les pages publiques, protegees et admin.
- La configuration providers (angular/src/app/app.config.ts) enregistre les services globaux.
- L'authentification globale (angular/src/app/services/auth.service.ts) gere session, token et profil.
- L'injection JWT (angular/src/app/interceptors/auth.interceptor.ts) securise automatiquement les requetes backend.

### 3.2 Backend (Express)

- Le point d'entree (x3com-backend/server.js) initialise dependances, middlewares et routes.
- Les middlewares de securite appliquent les controles transverses (auth, rate limit, CORS, headers).
- Les endpoints REST sont organises par domaine: authentification/compte, commandes/paiement, RDV, admin/diagnostic/stats, contenus pedagogiques.

## 4. Sécurité et contrôle d'accès

Extrait code Validation de variable critique au démarrage:

```
const JWT_SECRET = process.env.JWT_SECRET;

if (!JWT_SECRET) {
  console.error('\n✗ JWT_SECRET manquant dans .env — le serveur ne peut pas démarrer en sécurité.');
  process.exit(1);
}
```

Extrait code Middleware JWT:

```
function requireAuth(req, res, next) {
  const auth = req.headers['authorization'];
  if (!auth || !auth.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Token manquant — authentification requise.' });
  }
  try {
    req.user = jwt.verify(auth.slice(7), JWT_SECRET);
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Token invalide ou expire. Reconnectez-vous.' });
  }
}
```

Extrait code Autorisation propriétaire/admin:

```
function requireOwnerOrAdmin(req, res, next) {
  requireAuth(req, res, (err) => {
    if (err) return next(err);
    const targetId = parseInt(req.params.id);
    if (req.user?.role === 'admin' || req.user?.id === targetId) {
      return next();
    }
    return res.status(403).json({ error:'Accès refusé—vous ne pouvez modifier que votre propre compte.'});
  });
}
```

- Helmet ajoute des en-tetes HTTP de securite, dont une CSP stricte.
- Le rate limiting sur auth/contact reduit les attaques par brute force et abus d'API.
- Le filtrage CORS limite les origines autorisees aux domaines frontend attendus.

## 5. Authentification et session

### 5.1 Login/Register

Cette sous-partie couvre le cycle d'acces principal: creation de compte puis connexion securisee.

- POST /login authentifie un utilisateur existant.
- POST /register cree un compte client.
- Validation email backend avant insertion.
- Mot de passe minimum 8 caracteres.
- JWT de session valide 7 jours.

Extrait code Login avec migration automatique des anciens hash:

```
app.post('/login', limiterAuth, async (req, res) => {
  const { email, motDePasse } = req.body;
  const { data: rows } = await supabase.from('utilisateurs').select('*').eq('email', email).limit(1);
  const row = rows?.[0];
  const hash = row?.motdepasse ?? '';

  let valide = false;
  if (hash.startsWith('$2'))           valide = await bcrypt.compare(motDePasse, hash);
  else if (hash.startsWith('pbkdf2$')) valide = await verifierPBKDF2(motDePasse, hash);
  else valide = createHash('sha256').update(motDePasse).digest('hex') === hash;

  if (!valide) return res.status(401).json({ error: 'Identifiants incorrects' });

  if (!hash.startsWith('$2')) {
    const h2 = await bcrypt.hash(motDePasse, 12);
    await supabase.from('utilisateurs').update({ motdepasse: h2 }).eq('id', row.id);
  }

  const token = jwt.sign({ id: row.id, role: row.role }, JWT_SECRET, { expiresIn: '7d' });
  return res.json({ utilisateur: utilisateurToAngular(row), token });
});
```

Comment ca marche: le serveur verifie le mot de passe selon son format (bcrypt, PBKDF2 ou legacy), migre vers bcrypt si necessaire, puis retourne un JWT de session.

### 5.2 Refresh token applicatif

Le refresh applicatif permet de prolonger la session sans demander une reconnexion complete a chaque ouverture de l'application.

Endpoint: POST /refresh-token

```
app.post('/refresh-token', limiterAuth, async (req, res) => {
  const { token } = req.body;
  if (!token) return res.status(400).json({ error: 'Token requis' });

  const decoded = jwt.verify(token, JWT_SECRET, { ignoreExpiration: true });
  // vérifie utilisateur puis régénère un token 7j
});
```

Bootstrap frontend:

```
export function initialiserAuthentification(authService: AuthService) {
  return () => {
    if (!authService.getToken()) return;
    authService.refreshToken().subscribe();
  };
}
```

Interceptor frontend:

```
if (token && estRequeteBackend) {
  const reqAuth = req.clone({
    setHeaders: { Authorization: `Bearer ${token}` }
  });
  return next(reqAuth);
}
```

Extrait code Refresh token cote frontend:

```
refreshToken(): Observable<string | null> {
  const token = this.getToken();
  if (!token) return of(null);

  return this.http.post<{ token: string }>(`${this.backend}/refresh-token`, { token }).pipe(
    map((response) => response.token ?? null),
    tap((newToken) => {
      if (newToken) localStorage.setItem(TOKEN_KEY, newToken);
    }),
    catchError(() => {
      this.deconnecter();
      return of(null);
    })
  );
}
```

Comment ca marche: l'interceptor ajoute le header Authorization uniquement pour le backend X3COM, le refresh prolonge la session au chargement, et tout echec invalide la session locale.

## 6. Gestion compte utilisateur

La gestion de compte couvre les operations de consultation, modification et suppression, avec un controle d'acces strict.

Endpoints principaux:

- PATCH /utilisateurs/:id/password met a jour le mot de passe.
- PATCH /utilisateurs/:id met a jour les informations de profil.
- DELETE /utilisateurs/:id supprime le compte utilisateur.
- GET /utilisateurs/:id recupere un profil precis.
- GET /utilisateurs/recherche?q=... recherche des comptes selon un critere.
- GET /utilisateurs (admin) retourne la liste globale.

Garanties techniques:

- Controle proprietaire/admin sur les actions sensibles.
- Protection contre les acces de type IDOR.
- Normalisation des champs avant persistance pour eviter les donnees incoherentes.

## 7. Mot de passe oublié et réinitialisation

Le flux de recuperation de mot de passe permet a un utilisateur de reprendre l'acces a son compte sans intervention manuelle du support.

Endpoints exposes:

- POST /forgot-password declenche la generation d'un lien de reinitialisation.
- POST /reset-password valide le token et enregistre le nouveau mot de passe.

Mecanisme de securite:

- Generation d'un token aleatoire (32 bytes).
- Stockage en base dans password_resets avec date d'expiration.
- Duree de validite limitee a 60 minutes.
- Marquage used_at apres usage et purge des tokens obsoletes.
- Envoi du lien via Resend pour conserver une tracabilite transactionnelle.

## 8. Commandes, paiement, facture PDF

Cette section decrit le parcours technique complet de l'achat, du checkout Stripe jusqu'a la facture telechargeable.

### 8.1 Commandes

- GET /commandes retourne la liste visible par l'utilisateur courant.
- GET /commandes/:id retourne le detail d'une commande precise.
- PATCH /commandes/:id (admin) modifie le statut ou les attributs de suivi.
- DELETE /commandes/:id (admin) supprime une commande selon la politique metier.
- POST /commandes/:id/annuler traite l'annulation et ses effets metier.

### 8.2 Paiement Stripe

- POST /create-checkout-session prepare une session Stripe securisee.
- GET /session/:sessionId verifie l'etat d'une session de paiement.
- POST /webhook recoit les evenements Stripe signes cote serveur.

```
const session = await stripe.checkout.sessions.create({
  payment_method_types: ['card'],
  mode: 'payment',
  line_items: [...],
  success_url:`${process.env.FRONTEND_URL||'http://localhost:4200'}/commande?session_id={CHECKOUT_SESSION_I
  cancel_url: `${process.env.FRONTEND_URL||'http://localhost:4200'}/paiement?annule=1`,
});
```

### 8.3 Webhook Stripe

- Vérification signature obligatoire
- Création commande en base après paiement confirmé
- Génération numero_commande
- Envoi des emails de confirmation
-

Extrait code Validation du webhook et insertion commande:

```
app.post('/webhook', async (req, res) => {
  const sig = req.headers['stripe-signature'];
  const secret = process.env.STRIPE_WEBHOOK_SECRET;
  const event = stripe.webhooks.constructEvent(req.body, sig, secret);

  if (event.type === 'checkout.session.completed') {
    const session = event.data.object;
    const meta = session.metadata || {};

    const { data: created } = await supabase.from('commandes').insert({
      utilisateurid: parseInt(meta.utilisateurId) || null,
      offreid: parseInt(meta.offreId),
      statut: 'paiement_confirme',
      prix: parseFloat(meta.prix),
      notes: meta.nomOffre,
      stripesessionid: session.id,
      datepaiement: new Date().toISOString(),
    }).select('id').single();

    const numeroCommande = creerNumeroCommande(created.id, new Date().toISOString());
    await supabase.from('commandes').update({ numero_commande: numeroCommande }).eq('id', created.id);
  }

  return res.sendStatus(200);
});
```

Comment ca marche: Stripe notifie le serveur, la signature est verifiee, puis les metadata sont converties en commande metier en base avec un numero de commande genere apres insertion.

### 8.4 Facture PDF

La facture est generee a la demande cote serveur pour garantir l'integrite des montants et des informations client.

Endpoint: GET /commandes/:id/pdf

```
const doc = new PDFDocument({ size: 'A4', margin: 48 });
res.setHeader('Content-Type', 'application/pdf');
res.setHeader('Content-Disposition', `attachment; filename="facture-${numeroCommande}.pdf"`);
doc.pipe(res);
```

## 9. Prise de rendez-vous (RDV)

Le module RDV gere la disponibilite des creneaux, la creation des demandes et le traitement administratif des statuts.

- GET /rdv/creneaux-pris retourne les creneaux deja reserves pour une date donnee.
- GET /rdv/creneaux-disponibles retourne uniquement les creneaux encore libres.
- POST /rdv cree une nouvelle demande de rendez-vous.
- GET /rdv (admin) affiche l'ensemble du planning.
- GET /rdv/:id retourne le detail d'un rendez-vous.
- PATCH /rdv/:id/statut (admin) confirme ou annule la demande.
- DELETE /rdv/:id (admin) supprime un rendez-vous.

**Comportement métier:**

- Creneaux proposes de 09:00 a 17:30 avec un pas de 30 minutes.
- Blocage des week-ends au niveau backend.
- Blocage du creneau des la creation du RDV.
- Reservation temporaire desactivee (routes legacy conservees pour compatibilite).

```
const prises = new Set([...rdvs.map(r => r.heure)]);
const tousCreneaux = [];
for (let h = 9; h < 18; h++) {
  tousCreneaux.push(`${String(h).padStart(2, '0')}:00`);
  tousCreneaux.push(`${String(h).padStart(2, '0')}:30`);
}
const disponibles = tousCreneaux.filter(c => !prises.has(c));
```

Extrait code Creation RDV avec normalisation de date:

```
app.post('/rdv', async (req, res) => {
  const { nom, email, telephone, adresse, date, heure, service, rubrique, notes } = req.body;
  if (!nom || !email || !telephone || !date || !heure) {
    return res.status(400).json({ error: 'Champs obligatoires manquants.' });
  }

  const dateNormalisee = /^\d{4}-\d{2}-\d{2}$/.test(date)
    ? date
    : `${new Date(date).getUTCFullYear()}-${String(new Date(date).getUTCMonth() + 1).padStart(2, '0')}-${S

  const [y, mo, d2] = dateNormalisee.split('-').map(Number);
  const jourSemaine = new Date(y, mo - 1, d2).getDay();
  if (jourSemaine === 0 || jourSemaine === 6) {
    return res.status(400).json({ error: 'Les rendez-vous ne sont pas disponibles le week-end.' });
  }

  const { data } = await supabase.from('rdv').insert([{
    nom: nom.trim(), email: email.trim().toLowerCase(), telephone: telephone.trim(),
    adresse: adresse?.trim() || '', date: dateNormalisee, heure,
    service: service || 'diagnostic', rubrique: rubrique || '', notes: notes?.trim() || '',
    statut: 'en_attente',
  }]).select().single();

  return res.status(201).json(data);
});
```

Comment ca marche: les donnees sont nettoyees, la date est normalisee pour eviter les effets timezone, la regle no-weekend est appliquee, puis le RDV est cree en statut en_attente.

```
this.http.get<string[]>(`${environment.backendUrl}/rdv/creneaux-disponibles`, {
  params: { date: this.dateSelectionnee },
}).subscribe({
  next: (data) => {
    this.creneauxDisponibles = data ?? [];
  }
});
```

## 10. Module admin

Le module administrateur centralise les actions de pilotage operationnel et de supervision de la plateforme. Toutes les routes de cette section sont protegees par un controle de role admin. Les routes principales sont les suivantes:

- GET /stats fournit une vue agregee de l'activite (commandes, RDV, utilisateurs, offres).
- GET /diagnostic verifie la sante technique des dependances backend.
- GET /utilisateurs donne une vision globale des comptes pour la gestion support.
- Le CRUD des offres permet de creer, modifier, ordonner et supprimer les prestations commerciales.
- Les routes RDV (GET /rdv, PATCH /rdv/:id/statut, DELETE /rdv/:id) permettent de traiter le planning.
- Les routes commandes admin permettent de suivre les statuts et de gerer les operations de back-office.

```
app.get('/stats', requireAdmin, async (req, res) => {
  // commandes, rdv, utilisateurs, offres populaires
  res.json(stats);
});
```

Extrait code Aggregation RDV par statut:

```
const { data: rdvs } = await supabase
  .from('rdv')
  .select('statut');

stats.rdv = {
  total: rdvs.length,
  statut: {
    en_attente: rdvs.filter(r => r.statut === 'en_attente').length,
    confirme: rdvs.filter(r => r.statut === 'confirme').length,
    annule: rdvs.filter(r => r.statut === 'annule').length,
  }
};
```

Comment ca marche: la route admin calcule les indicateurs directement depuis la base pour afficher un etat temps reel des RDV a traiter.

En pratique, cette section sert de centre de controle: elle expose l'etat courant de l'application et fournit les leviers d'action necessaires aux equipes d'exploitation.

## 11. Contenus Saviez-vous

Cette partie alimente les pages pedagogiques du site (glossaire technique, arret du cuivre, resilience) avec des donnees dynamiques depuis Supabase.

### 11.1 Endpoint GET /glossaire

```
app.get('/glossaire', async (req, res) => {
  try {
    const { data, error } = await supabase
      .from('glossaire')
      .select('id, terme, definition, lettre')
      .order('lettre', { ascending: true })
      .order('terme',  { ascending: true });

    if (error) throw error;
    res.json(data);
  } catch (err) {
    res.status(500).json({ message: 'Erreur serveur glossaire' });
  }
});
```

Explication: aucune entree obligatoire, lecture de la table glossaire avec tri alphabetique, puis retour d'un tableau JSON directement exploitable par le frontend.

```
[
  { "id": 1, "terme": "ADSL", "definition": "...", "lettre": "A" },
  { "id": 2, "terme": "Acces Internet", "definition": "...", "lettre": "A" }
]
```

### 11.2 Endpoint GET /communes/recherche

```
app.get('/communes/recherche', async (req, res) => {
  const q = req.query.q ? String(req.query.q).trim().toLowerCase() : '';

  if (!q || q.length < 2) {
    return res.status(400).json({ error: 'Minumum 2 caractères' });
  }

  try {
    const { data, error } = await supabase
      .from('communes_fermeture_cuivre')
      .select('*')
      .or(`commune.ilike.%${q}%,code_postal.ilike.%${q}%`)
      .limit(5);

    if (error) throw error;
    res.json(data || []);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

Explication: la route valide un parametre q (minimum 2 caracteres), cherche par nom de commune ou code postal, puis renvoie jusqu'a 5 resultats pour un affichage rapide en auto-completion.

### 11.3 Consommation frontend arret du cuivre

```
rechercher(query: string): Observable<Commune[]> {
  return this.http.get<Commune[]>(`${this.backend}/communes/recherche`, {
    params: { q: query }
  });
}
```

```
this.communesService.rechercher(this.communeQuery).subscribe({
  next: (communes) => {
    if (communes.length > 0) {
      this.resultat.set(communes[0]);
    } else {
      this.resultat.set(null);
      this.erreur = 'Commune non trouvée. Consultez l\'ARCEP pour plus d\'infos.';
    }
  },
  error: () => {
    this.erreur = 'Erreur lors de la recherche.';
  }
});
```

Explication: le service centralise l'appel HTTP, et le composant gere les etats de l'interface (resultat, aucun resultat, erreur serveur).

### 11.4 Consommation frontend glossaire

```
const response = await fetch(`${environment.backendUrl}/glossaire`, {
  signal: controller.signal
});

this.tousLesTermes = await response.json();
this.lettresDisponibles = [...new Set(this.tousLesTermes.map(t => t.lettre))].sort();
```

Explication: les termes sont charges en une fois, puis filtres localement pour la recherche texte et la navigation alphabetique. Le timeout evite de bloquer la vue en cas d'appel trop long.

## 12. Modèle de données (principales tables)

Le modele de donnees est structure pour couvrir les parcours clients (achat, compte, rendez-vous) et les contenus pedagogiques.
Les tables principales et leur role sont les suivants:

- utilisateurs stocke les comptes, les roles et les informations de profil utilisees par l'authentification et l'espace client.
- commandes stocke le cycle de vie des achats (statut, prix, session Stripe, numero de commande, dates de paiement/annulation).
- offres contient le catalogue commercial affiche dans l'interface (nom, prix, description, options, ordre d'affichage).
- rdv enregistre les demandes de rendez-vous (coordonnees, date/heure, service, statut operationnel).
- password_resets gere les tokens de reinitialisation et leur expiration.
- glossaire alimente la page pedagogique des termes techniques.
- communes_fermeture_cuivre alimente la recherche locale sur l'arret du cuivre.

Les scripts de migration et d'evolution du schema sont centralises dans x3com-backend/migration_supabase.sql.

## 13. Observabilité et diagnostic

Cette section regroupe les points d'entree qui permettent de verifier rapidement que la plateforme fonctionne correctement en production.
Les endpoints techniques sont:

- GET /health, utilise pour confirmer que le serveur repond et que le service est demarre.
- GET /diagnostic (admin), utilise pour controler plus finement l'etat des dependances critiques.

Concretement, ces verifications permettent:

- de valider l'etat de l'environnement d'execution;
- de confirmer la connectivite Supabase;
- de verifier la disponibilite des integrations Stripe et Resend;
- d'accelerer l'analyse en cas d'incident (erreur 5xx, probleme de paiement, email non envoye).

## 14. Flux critiques (résumé)

Les flux suivants representent les scenarios metier les plus sensibles. Ils doivent rester coherents apres chaque evolution du code.

### 14.1 Achat

Le flux d'achat transforme une intention client en commande confirmee et tracable:

1. L'utilisateur choisit une offre depuis le frontend.
2. Le frontend appelle POST /create-checkout-session.
3. Le client est redirige vers Stripe pour le paiement.
4. Stripe notifie le backend via le webhook checkout.session.completed.
5. Le backend insere la commande en base et genere un numero metier.
6. L'utilisateur retrouve la commande et peut telecharger la facture PDF.

### 14.2 RDV

Le flux RDV garantit une prise de rendez-vous coherente avec les disponibilites reelles:

1. L'utilisateur choisit une date dans l'interface.
2. Le frontend interroge GET /rdv/creneaux-disponibles.
3. L'utilisateur selectionne un creneau libre.
4. Le frontend envoie la demande via POST /rdv.
5. L'equipe admin confirme ou annule selon le contexte operationnel.

### 14.3 Récupération mot de passe

Le flux de recuperation garantit un acces restaure sans compromettre la securite du compte:

1. L'utilisateur lance POST /forgot-password avec son email.
2. Il recoit un lien temporaire contenant un token de reinitialisation.
3. Il soumet le nouveau mot de passe via POST /reset-password.

## 15. Points de vigilance techniques

Ces points sont critiques pour la securite, la conformite et la stabilite de l'application:

- La cle Supabase service_role ne doit jamais etre exposee cote client, car elle donne des privileges eleves.
- Les routes admin doivent rester protegees par requireAdmin pour eviter les acces non autorises.
- Le webhook Stripe doit systematiquement valider sa signature afin de rejeter les notifications falsifiees.
- Les erreurs CORS en production proviennent souvent d'une mauvaise configuration FRONTEND_URL/ALLOWED_ORIGINS.
- Les emails Resend necessitent des domaines correctement valides, sinon les notifications transactionnelles echouent.
- Les cles API et secrets (Supabase, Stripe, Resend, JWT) sont geres dans les variables d'environnement Render, jamais stockes en dur dans le depot.

Avant chaque mise en production, il est recommande de verifier explicitement ces cinq points dans une checklist de deploiement.

Rayan Saboun 18/05/2026

# Document technique